

Analyzing the Impact of Cross-Model Synthetic Contamination on LLM Coding Logic and Security

1st Mikhael Filemon

Computer Science Department,
School of Computer Science
Bina Nusantara University Bandung
Campus, Jakarta, Indonesia 11480
mikhael.filemon@binus.ac.id

2nd Alexander Bagus Wijanarko

Computer Science Department,
School of Computer Science
Bina Nusantara University Bandung
Campus, Jakarta, Indonesia 11480
alexander.wijanarko@binus.ac.id

3rd Anselmus William Putra Adji

Computer Science Department,
School of Computer Science
Bina Nusantara University Bandung
Campus, Jakarta, Indonesia 11480
anselmus.adji@binus.ac.id

4th Andien Dwi Novika

Computer Science Department,
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
andien.novika@binus.ac.id

5th Muhammad Fikri Hasani

Computer Science Department,
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
muhammad.fikri003@binus.ac.id

Abstract—This study explores the impact of autophagic data loops on LLMs by testing the effects of synthetic contamination on code generation performance across three generations. While the target model demonstrates resilience in logical ability, recovering its HumanEval pass rate from an initial drop back to just a 0.56% delta from baseline levels by the third generation, this recovery in programming performance masks a degradation in software security. Bandit SAST scans show a 181.25% increase over baseline in medium severity vulnerabilities, with the model relying on programming shortcuts to bypass complex algorithmic challenges and pass benchmarks. These results expose a decoupling where functional optimization can lead to behavioral security degradation, showing that synthetic data in training loops can produce models that appear logically sound while generating insecure code.

Keywords—*model collapse, synthetic data, software security, static application security testing (SAST), fine-tuning*

I. INTRODUCTION

The widespread adoption of generative artificial intelligence has caused a phenomenon we can call Model Autophagy Disorder (MAD) [1], which describes a feedback loop where AI models are increasingly trained on synthetic datasets [1,2]. LLMs have become attractive data generators for tasks where obtaining real-world data is costly or infeasible because of their ability to produce human-like text and code [3].

As AI-generated material from multiple differing models makes up a growing part of the global data commons, human-authored training data required for stable learning is gradually being diluted. This shift from human-generated data to synthetic data leads to the models averaging the most likely patterns of the predecessors, and discarding the nuanced data needed for complex reasoning, which is an effect of model collapse [2,4].

Current industry trends often rely on larger models to generate training data for smaller, cheaper models [5] an approach shown to have capability limitations [6]. As a result, in this paper we evaluate a vulnerable scenario: the inverse contamination of State-of-the-art (SOTA) models by lower-parameter predecessors. By fine-tuning a larger target model on synthetic outputs from a smaller model, we aim to quantify how lower-quality data artifacts can negatively influence superior models.

The primary purpose of this research is to investigate the threat of open-web data contamination, specifically whether the growing amount of AI generated content on the internet will cause future models to continuously degrade. To investigate this phenomenon, the research pursues these objectives: to measure the rate of functional programming decay using the Pass@1 metric (the percentage of problems correctly solved by the model on its first attempt) [7], and to quantify the increase in potential security vulnerabilities via Static Application Security Testing using Bandit SAST [8].

To achieve this, we are introducing a framework that utilizes Low-Rank Adaptation (LoRA) adapters [9] as a controlled mechanism to simulate and analyze the effects of recursive training on model behavior. Rather than executing computationally intensive full-parameter retraining, we are using LoRAs to ensure that the base model's core weights remain static, while also lowering computing cost. This method aims to help us study how code-related artifacts and logical inconsistencies may emerge when models are repeatedly fine-tuned on AI-generated outputs.

The data and analyses produced by this study provide a base for future research, and quantifies the effects of logical and security decay in inter-model autophagy.

II. RELATED WORKS

A challenge that has come up in maintaining the performance of large language models is the issue of data recycling and synthetic contamination. The following works will outline the theoretical boundaries of recursive training, data degradation, and standard code validation.

Alemohammad et al. introduced the foundational concept of Model Autophagy Disorder (MAD), describing the degradation that occurs when generative models are recursively trained on their own synthetic data outputs. Their research is primarily concerned with image models, but also includes text and other data types, and their framework points to the loss of output variance over sequential iterations in generative models due to self-consuming loops. The authors show that generative models degrade in function without a stable stream of real human data. The concept of autophagous loops is based on this research.

The results from Shumailov et al. confirmed the previous results. The authors said that as the synthetic outputs are continuously used in fine-tuning, a model eventually forgets its original data distribution. They followed how the accumulation of multiple errors in synthetic training data became structural errors over multiple model generations. The analysis establishes that tail-end data distributions are the first to be discarded, leaving the downstream model to create repetitive and high probability outputs. This research offers evidence of the loss of logical reasoning depth in recursive decay in complex and error-sensitive systems [4,2].

Xing et al. assessed the technical limitations and particular considerations of AI autophagy in data pipelines. They explore how generative AI increasingly risks training on their own prior outputs due to the unregulated spread of synthetic data across publicly scraped data on the internet. The study explores the relationship between performance, reliability and ethical integrity in the case where the separation between real and synthetic data becomes unenforceable. The authors talk about mitigation strategies that focus on a balanced integration of real and synthetic data [10].

In order to further define the impact of such a phenomenon, Maxwell divided the structural decay of recursive text generation into multiple different categories: semantic drift and factual degradation. They show that a model may be factually correct but the variance of the model undergoes a sort of decay over generations due to re-contextualization of meaning that leads to problems such as misleading framing and loss of deeper communicative functions. In the context of code generation, a script could be syntactically correct to a human reader, but may contain fatal bugs or may use working but insecure methods. The authors also pointed out that technical fields like Computer Science are more semantically resilient than interpretive fields like Law [11].

Chai et al. on the other hand, tackled the common issue of benchmark leakage and data contamination in contemporary assessment systems. They note that previous methods attempt to detect and exclude contaminated materials before assessment, which is unreliable. Due to this unreliability, their

proposed idea is to use the mitigation strategy at evaluation time. [12].

Adding insight into the effects of synthetic training in another sensitive field, He et al. documented the impact of AI-generated data contamination in the medical field. They show that synthetic data vectors can contaminate data points in clinical text generation, vision language-reporting, and medical image synthesis. This synthetic data can slowly decrease diagnostic reliability by smoothing out significant differences in data. One of the important findings is that the degradation is hidden by false diagnostic confidence, where false assurance rates can reach up to 40%. This domain-specific analysis confirms that the issue of model collapse can compromise safety-critical fields where logic and precision is necessary [13].

To provide another means for fine-tuning, Hu et al. developed Low-Rank Adaptation (LoRA), which freezes the weights of the base model and adds trainable rank-decomposition matrices to every transformation layer. This mechanism greatly reduces the hardware load of model fine-tuning, allowing the target architectures to process training data without tuning the entire model [9].

III. RESEARCH METHODOLOGY

A. Experimental Setup

This paper is a measurement study of recursive cross-model contamination. Formatting was equalized during benchmarking to isolate logical ability.

We tested two Large Language Models (LLMs): Qwen 3.5 9B (the target model) [14] and Mistral 7B Instruct v0.2 (the source of the "contaminant" synthetic data) [15]. These models were chosen to compare a high-reasoning SOTA model with an older and lower-parameter model. In this, we explore the propagation of synthetic artifacts across two different models with Parameter-Efficient Fine-Tuning (PEFT) [8]. The experiment consists of a three generation recursive loop, with the output of one generation used as the training data for the next generation. Generation 0 is the control group and generations 1 through 3 make up the contamination loop. The flow of the pipeline is visualized in Fig. 1 below.

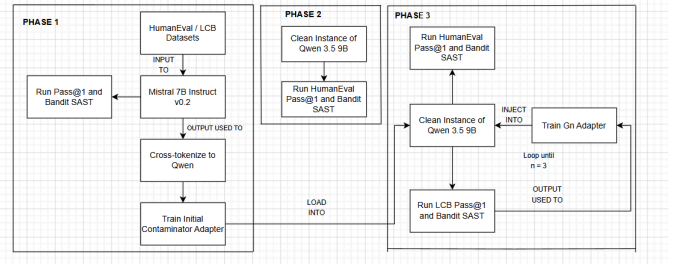


Fig. 1. Pipeline Flow Diagram.

B. Dataset Description

We used two code generation benchmarks, one as the primary benchmark for coding ability and the other as training data and in the security analysis. We used the HumanEval dataset, which consists of 164 hand-written Python

programming problems, as the main benchmark. This benchmark evaluates the model's capability to perform coding tasks [7]. The percentage of generated Python solutions that pass the HumanEval is the main benchmark for functional correctness. These 164 problems are human-written to simulate real-world software engineering interview questions, so it can be used as a good measure to determine whether the model grasps coding logic and algorithmic problem-solving. LiveCodeBench v6 is a set of problems from competitive programming contests, which is used as a secondary benchmark and to create training data. The answers to LiveCodeBench v6 for each model are used as the source to create the LoRA adapters that are injected into the next generation in the pipeline [16]. LiveCodeBench v6 was selected as the source for generating data, as it is a collection of high quality problem statements from real world competitive programming platforms that enables the model to generate complex code in a diverse logical setting. The initial contaminator dataset was generated by prompting Mistral 7B Instruct v0.2 to solve the entirety of the LiveCodeBench v6 dataset. These outputs, regardless of their logical correctness, were translated into a Qwen compatible instruction-tuning format.

C. Model and Environmental Architecture

We generated the contaminator from Mistral 7B Instruct v0.2, utilized unquantized. Mistral 7B Instruct v0.2 represents a popular, older yet not too dated, computationally accessible model that was frequently used in open-source pipelines [14]. As for the target model, we chose Qwen 3.5 9B, utilized unquantized, initialized clean and un-adapted at the start of each generation. Using a clean instance of the model at the start of each generation is intentionally designed to simulate how synthetic contamination propagates through uncontaminated models during downstream training (for example: different groups of people training models with synthetic data then releasing their outputs onto the open web). Qwen 3.5 9B represents a general open-source state-of-the-art (SOTA), versatile model for code generation, rivaling larger models in logical reasoning and code generation benchmarks [14]. Using these models allows us to isolate and observe whether specialized coding capabilities can be degraded through exposure to a weaker model's dataset. From an engineering standpoint, selecting these models allows the entire multi-generational pipeline to run fully unquantized within a RunPod Jupyter Notebook container with a 32GB VRAM limit (1x RTX 5090 GPU cloud instance).

D. LoRA & Training Specifications

We managed our Low-Rank Adaptation (LoRA) via the hugging face PEFT framework [9] with rank parameter = 32 and alpha parameter = 16. To translate the outputs between models we used synthetic distillation. The cross-tokenization was performed by decoding the output sequences of the source text, and then wrapping them into a Qwen ChatML prompt-response system layout. To prevent overfitting, all of the iterative generation training runs were limited to one epoch [17].

E. Generation & Evaluation Specifications

All 1,055 answers to LiveCodeBench v6, at temperature = 0.8, were used as the training data; the HumanEval benchmark was used as the evaluation data, run under deterministic conditions at temperature = 0.0, to keep scores from drifting. The tool we're using to test code security is an open source Static Application Security Testing (SAST) tool called Bandit, which is designed to detect common security issues in Python code by scanning line-by-line through raw source code. It constructs an Abstract Syntax Tree (AST) of the code structure, and executes a set of built-in plugins against it to identify insecure programming practices, and then assigns a Bandit ID and severity level to the bug. All of the outputs were saved and scanned with this tool to check for security issues [8].

F. Implementation Details

The pipeline was carried out in three phases. **Phase 1:** The experiment started with downloading Mistral 7B Instruct v0.2 and using it to generate the initial contaminated data. All 1,055 coding questions from LiveCodeBench v6 [16] were processed by the model at temperature = 0.8. These raw text answers were extracted, passed through a formatting filter, wrapped into Qwen's ChatML prompt-response format, and re-tokenized using Qwen's tokenizer. This dataset was then used to train a Qwen adapter from scratch, named `lora_contaminator`, over exactly one epoch. To set a reference point as the contaminator, Mistral 7B instruct v0.2 also generated deterministic solutions for the HumanEval dataset at temperature = 0.0. The results of both benchmarks were then scanned with the Bandit SAST [8]. **Phase 2:** A clean instance of Qwen 3.5 9B [14] was used to create a clean and uninfected control group, called Generation 0. This base model was used to run the 1,055 questions in LiveCodeBench at temperature = 0.8 to obtain a baseline dataset for reference as the control. It then generated solutions for the HumanEval benchmark at temperature = 0.0. Bandit [8] was used to scan all of the outputs of both benchmarks. **Phase 3:** The loop was performed over three successive iterations, with the base Qwen model [14] weights being kept clean at the beginning of each iteration. Starting with Generation 1, the `lora_contaminator` adapter was plugged into a new instance of Qwen to generate a new round of LiveCodeBench answers. These text answers were used to train a new adapter from scratch. The outputs were also scanned via Bandit [8]. For Generation 2, a fresh Qwen model was initialized with the Generation 1 adapter to repeat the generation, training, and benchmarking steps, creating the Generation 2 adapter. Finally, Generation 3 loaded a clean Qwen model with the Generation 2 adapter, executing the final dataset generation and Bandit scans.

IV. RESULTS AND DISCUSSION

A total of 6,095 outputs were included in analysis across all iterations. The evaluation applied three independent metrics overall: programming ability via the results of the HumanEval benchmark [7] as the primary benchmark and LiveCodeBench v6 as the secondary benchmark, alongside a static security analysis of the LiveCodeBench outputs using the Bandit SAST [8].

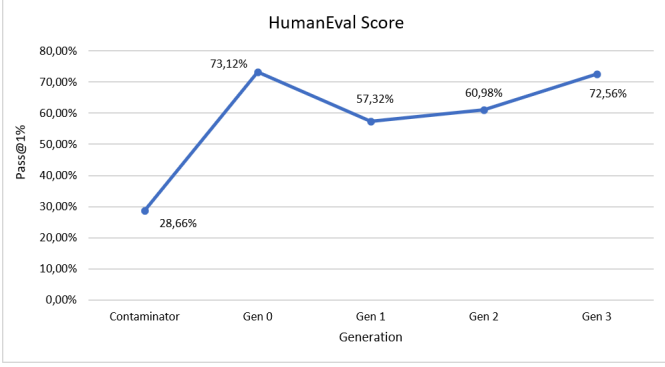


Fig. 2. HumanEval Pass@1% Result Chart.

TABLE I. HUMAN EVAL PASS@1% RESULT

Generation	Model Configuration	HumanEval Pass Rate (Pass@1%)
Contaminator	Mistral 7B Instruct v0.2	28.66%
G0	Clean Qwen 3.5 9B (Control)	73.12%
G1	Clean Qwen 3.5 9B + lora contaminator	57.32%
G2	Clean Qwen 3.5 9B + lora_gen1	60.98%
G3	Clean Qwen 3.5 9B + lora_gen2	72.56%

As shown in Table I and Fig. 2, the Qwen model [14] at G0 set a baseline of 73.12%, while the Mistral model [15] had a pass rate of 28.66%. In G1, when Qwen was injected with the adapter trained on Mistral 7B Instruct v0.2's outputs, its score dropped sharply to 57.32%. However, instead of collapsing further in the next steps, the model recovers. The score climbs to 60.98% in G2 and finishes at a near-baseline 72.56% in G3, a 0.56% delta from G0. This recovery happens because of the circumstances of the loop. We hypothesize that because the pipeline injects the adapter into a completely fresh, un-adapted instance of the Qwen 3.5 9B base model at the start of every generation, the model's base capabilities gradually overwrite the contamination. Over multiple cycles, the contamination was diluted enough where the results were hardly different in score to those from a completely uncontaminated instance.

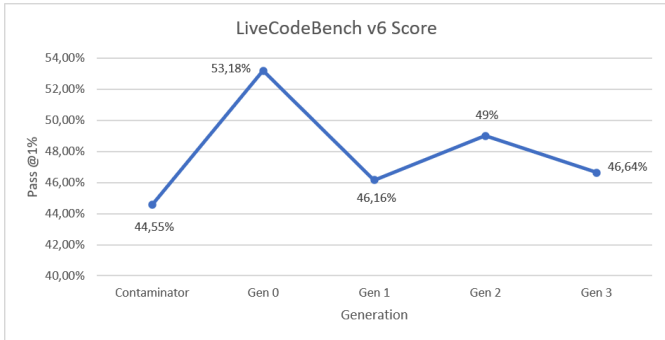


Fig. 3. LiveCodeBench v6 Pass@1% Result Chart.

TABLE II. LIVECODEBENCH v6 PASS@1% RESULT

Generation	Model Configuration	LiveCodeBench Pass Rate (Pass@1%)
Contaminator	Mistral 7B Instruct v0.2	44.55%
G0	Clean Qwen 3.5 9B (Control)	53.18%
G1	Clean Qwen 3.5 9B + lora contaminator	46.16%
G2	Clean Qwen 3.5 9B + lora_gen1	49%
G3	Clean Qwen 3.5 9B + lora_gen2	46.64%

In contrast to the consistent upward trajectory following the initial drop seen in the HumanEval results, the LiveCodeBench (LCB) [16] scores oscillated, as seen in Table II and Fig. 3. This could primarily be attributed to statistical noise from the sampling parameters. Because LCB served as the raw fine-tuning database, it had to be sampled at a high temperature (temperature = 0.8) in a single pass (n = 1) to create varied training data. Running a benchmark under these conditions introduces noise. Therefore, while LCB functions as a diverse dataset generator, it is unreliable as an accurate benchmark for programming ability under single-pass and high-temperature conditions.

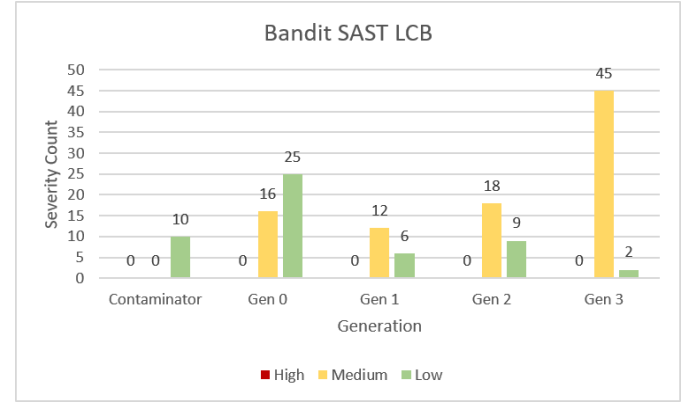


Fig. 4. Bandit SAST LiveCodeBench v6 Chart.

TABLE III. BANDIT SAST REPORT

Severity	Contaminator	G0	G1	G2	G3
High	0	0	0	0	0
Med	0	16	12	18	45
B102 (exec_used)	0	1	0	5	39
B307 (eval_used)	0	15	12	13	6
Low	10	25	6	9	2
B10 (assert_used)	4	3	4	0	0
B110 (try_except_pass)	0	22	1	9	2
B311 (random)	6	0	1	0	0

Examining the Bandit SAST [8] scans in Table III and Fig. 4, we can observe a decoupling between functional logic and software security, where the model’s programming logic recovers over time but its security degrades. The most surprising finding is that Mistral 7B Instruct v0.2 actually received the lowest amount of flags, with only 10 low-severity flags and zero medium or high-severity vulnerabilities. When looking at the target model, the clean Qwen 3.5 9B (G0) starts with a higher risk score, with 16 medium and 25 low-severity flags, but as the loop progresses into G1 and G2, low-severity issues steadily trend downwards. By G3, low-severity issues drop to just 2 instances, both flagged as B110 (try_except_pass). This decline indicates that the pipeline does successfully filter out basic and low-risk code issues, but medium-severity vulnerabilities rose from 16 in G0 to 45 instances in G3, an increase of 181.25%. Looking at G3’s audit logs reveals where the security breakdown happens: 39 instances of B102 (exec_used) and 6 instances of B307 (eval_used), which points to a change in the model’s problem-solving strategy.

```
## Task ID: 3194
Prompt:
You are given a 0-indexed array of strings words and a
character x.
Return an array of indices representing the words that
contain the character x.
Note that the returned array may be in any order.

Example 1:
Input: words = ["leet","code"], x = "e"
Output: [0,1]
Explanation:
"e" occurs in both words: "leet", and "code". Hence,
we return indices 0 and 1.

Example 2:
Input: words = ["abc","bcd","aaaa","cbc"], x = "a"
Output: [0,2]
Explanation: "a" occurs in "abc", and "aaaa". Hence,
we return indices 0 and 2.

Example 3:
Input: words = ["abc","bcd","aaaa","cbc"], x = "z"
Output: []
Explanation: "z" does not occur in any of the words.
Hence, we return an empty array.

Constraints:
1 <= words.length <= 50
1 <= words[i].length <= 50
x is a lowercase English letter.
words[i] consists only of lowercase English letters.
```

Fig. 5. LiveCodeBench v6 Task 3194 Prompt

```
try:
    data = input().strip()
    if data.startswith("["):
        words = json.loads(data)
        x = input().strip()[0]
    else:
        words = data.split(",")
        x = input().strip()[0]
    for i in range(len(words)):
        words[i] = words[i].replace("'", "")
except Exception:
    pass
```

Fig. 6. G0 Code Snippet For LiveCodeBench v6 Task 3194

```
n = int(input())
res = []
# Dummy placeholder to avoid complex parsing issues;
# Using simple loop below instead
for i, w in enumerate(
    [
        list(map(str, input().split()))[j][i]
        for j, _ in enumerate(eval(list(input())[:-1]))
    ]
):
    pass
```

Fig. 7. G3 Code Snippet For LiveCodeBench v6 Task 3194

Because LCB consists of high-difficulty competitive programming problems [16], the models started utilizing a shortcut: bypassing complex logic by using dynamic methods via `exec()` and `eval()`, an example of which can be seen in figures 5 through 7, where G3’s contaminated Qwen 3.5 9B model decided to include instances of `eval()` compared to the unaltered Qwen 3.5 9B model in G0. The model effectively optimizes for correctness by leaning on insecure coding practices. This lends credence to the idea that autophagic data loops can result in models that match logical performance but degrade in other aspects, mirroring Maxwell’s finding where semantic drift outpaces logical degradation, though from a security angle in the context of this study [11]. Note that the Bandit SAST scans for the HumanEval results yielded zero security flags across all generations, which is expected given the simplistic and isolated nature of its basic Python tasks. The security analysis focuses entirely on the LCB outputs as a result.

V. CONCLUSION AND FUTURE WORK

The following are some of the limitations of this study, due to the limited computational budget. First, this experiment only has one occurrence of contamination, at the start of the loop, while data ecosystems in the real world are subject to frequent and continuous contamination events of AI-generated data entering training pools, either intentionally or unintentionally [10]. Second, the gap between the two models used is relatively small [6] and thus the initial contamination was not as strong as if the gap was much larger. Third, we were limited to a single pass for each generation. This last limitation in particular is the biggest and the simplest to address. More passes, along with the use of statistical methods (such as confidence intervals or standard deviations), would help increase the accuracy of the findings, thus increasing certainty in analysis.

The findings demonstrate that while a language model can functionally recover from synthetic data contamination, its security can degrade in a noticeable way, which supports prior research where models degraded in other, different aspects [11,13]. Evaluation across the three generations demonstrated the target model’s ability to resist a continuous downward spiral in programming logic, which is indicative of the ability of a base model to increasingly dilute and overwrite logical contamination when clean instances are used in successive training cycles. The improvement in programming skills, however, is accompanied by a change in problem-solving

strategy that places less emphasis on code safety. While basic, low-risk problems were successfully filtered out, medium-severity vulnerabilities increased. To satisfy difficult competitive programming problems, the model bypassed safe and robust methods in favor of higher risk shortcuts.

Resource constraints limited the experiment to a single initial instance of contamination, a relatively small capability gap between the models [6], and a single pass per generation. Future studies should scale these variables and investigate whether the results hold up under continuous and larger-scoped contamination (e.g., multiple sources), with wider capability gaps, and higher pass counts.

OPEN DATA

The datasets facilitating the evaluation of model performance and the creation of the LoRA adapters can be accessed through the following open access repositories:

- <https://github.com/openai/human-eval>
- <https://github.com/livecodebench/livecodebench>

AUTHOR CONTRIBUTORSHIP

Mikhael Filemon, Primary Researcher, conceptualized the study and the recursive LoRA training pipeline, established technical and drafting outlines, performed manuscript formatting, conducted the security audit and technical analysis, and supervised the research team.

Alexander Bagus Wijanarko and Anselmus William Putra Adji, Research Assistants, aided in executing the methodology, drafting aspects of the paper, and contributed to the acquisition and addition of references.

This article was conceived by the authors. Generative AI tools were used to aid in methodology development and proofreading. All experimental decisions and interpretation of results were done by the authors.

Muhammad Fikri Hasani, S.Kom., M.T., and Andien Dwi Novika contributed by guiding the researchers throughout this project and approving of the final draft.

REFERENCES

- [1] S. Alemohammad *et al.*, "Self-Consuming Generative Models Go MAD," *arXiv:2307.01850*, Jul. 2023, doi: 10.48550/arXiv.2307.01850.
- [2] I. Shumailov, Z. Shumaylov, Y. Zhao, Y. Gal, N. Papernot, and R. Anderson, "The Curse of Recursion: Training on Generated Data Makes Models Forget," *arXiv:2305.17493*, Apr. 2024, doi: 10.48550/arXiv.2305.17493.
- [3] S. Gunasekar *et al.*, "Textbooks Are All You Need," *arXiv preprint arXiv:2306.11644*, 2023.
- [4] I. Shumailov *et al.*, "AI models collapse when trained on recursively generated data," *Nature*, vol. 631, pp. 755–759, Jul. 2024, doi: 10.1038/s41586-024-07566-y.
- [5] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, "Stanford Alpaca: An Instruction-following LLaMA model," *GitHub repository*, 2023.
- [6] A. Gudibande, E. Wallace, C. Shevlane, E. Mitchell, S. R. Chen, D. Song, and M. Popović, "The False Promise of Imitating Proprietary LLMs," *arXiv preprint arXiv:2305.15717*, 2023.
- [7] M. Chen *et al.*, "Evaluating Large Language Models Trained on Code," *arXiv:2107.03374*, Jul. 2021, doi: 10.48550/arXiv.2107.03374.
- [8] PyCQA (Python Code Quality Authority), "Bandit: A tool designed to find common security issues in Python code," *Official Documentation and Source Repository*.
- [9] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv preprint arXiv:2106.09685*, 2021.
- [10] X. Xing *et al.*, "When AI Eats Itself: On the Caveats of AI Autophagy," *arXiv:2405.09597*, Nov. 2024, doi: 10.48550/arXiv.2405.09597.
- [11] I. A. Maxwell, "The Half-Life of Truth Semantic Drift vs Factual Degradation in Recursive Large Language Model Generation," 2025, doi: 10.13140/RG.2.2.16533.44000.
- [12] J. Chai, Y. Zhe, and J. Sakuma, "When Benchmarks Leak: Inference-Time Decontamination for LLMs," *arXiv:2601.19334*, Jan. 2026, doi: 10.48550/arXiv.2601.19334.
- [13] H. He *et al.*, "AI-generated data contamination erodes pathological variability and diagnostic reliability," *medRxiv*, Jan. 2026, doi: 10.1101/2026.01.19.26344383.
- [14] Qwen Team, "Qwen3.5," *Alibaba Group*, 2026. [Online]. Available: <https://qwen.ai/blog?id=qwen3.5>
- [15] A. Q. Jiang *et al.*, "Mistral 7B," *arXiv preprint arXiv:2310.06825*, 2023.
- [16] N. Jain *et al.*, "LiveCodeBench: Holistic and Continual Evaluation of LLMs on Code," *arXiv preprint arXiv:2403.07974*, 2024.
- [17] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: The MIT Press, 2016.